PLANIFICACIÓN SCRUM (9 SPRINTS) Y PRODUCT BACKLOG

Proyecto: Sistema de Gestión para la Liga Municipal de Morelia A.C.
Duración: 9 Semanas (Sprints de 1 semana) → 11/Diciembre
Metodología y Ceremonias:

- **Sprint Planning:** Lunes, 1 hora para asignar las historias del backlog y definir tareas técnicas en Jira/Trello
- **Daily Scrum:** 15 minutos diarios para sincronización de los 4 desarrolladores,
- **Sprint Review & Retrospective:** Viernes, demostración funcional del incremento al Product Owner/Cliente y retrospectiva técnica
- **Roles del Equipo:**
    - [Dev 1] PO / Frontend Lead;
    - [Dev 2] Scrum Master / Backend Lead;
    - [Dev 3] Lógica de Negocio / Algoritmos;
    - [Dev 4] QA, Documentos y Despliegue.

Parte 1: Product Backlog Estructurado (Épicas)

El backlog se organiza en las siguientes Épicas, mapeadas directamente a los Requisitos Funcionales (RF) y Áreas Funcionales (AF) del ERS:

- **Épica 1: Core, Seguridad y Gobernanza (AF-01, AF-02, RF-01, RF-02, RF-12)** - Autenticación, RBAC (Spatie), Auditoría Inmutable, Gestión de Temporadas, Convocatorias versionadas, Categorías y Grupos.
- **Épica 2: Afiliación y Control de Padrón (AF-03, RF-03, RF-04)** - Clubes, Equipos, Identidad de Personas (INE/CURP), Tarjetón Digital, Regla restrictiva de "Salto de 1 Categoría".
- **Épica 3: Logística y Calendarización (AF-04, AF-05, AF-06, RF-05, RF-06, RF-07)** - Catálogo de 40+ Canchas (U.D.C.), Motor de Fixture Automático, Ajustes y Programación Semanal.
- **Épica 4: Operación de Juego y Resultados (AF-07, AF-08, RF-08, RF-09)** - Cédula Arbitral Móvil (UI táctil), Captura de Resultados, y Cálculo Automatizado de Tablas de posiciones y goleo.
- **Épica 5: Régimen Disciplinario (AF-08, RF-10)** - Tribunal de Penas, Catálogo de Artículos, Boletín de Castigados, Inconformidades, y Regla de "Baja por 5 Defaults".
- **Épica 6: Finanzas y Administración (AF-10, AF-14, RF-13, RF-14)** - Tesorería, Aportaciones (Inscripción, Arbitraje), Control de Pagos, Asambleas y Acuerdos de delegados.
- **Épica 7: Fases Finales (AF-09, RF-11)** - Liguilla (Generación de Llaves), Ascensos, Descensos y Campeón de Campeones.
- **Épica 8: Difusión y Portal Público (AF-11, AF-12, AF-13, RF-12, RF-15, RF-16)** - Portal web de Aficionados, Notificaciones por Correo Electrónico y Exportables oficiales PDF/Imagen.

Parte 2: Planeación Detallada de los 9 Sprints

Sprint 1: Arquitectura Base, Seguridad y Estructura Deportiva (RF-01, RF-02, RF-12)

- **Objetivo (Sprint Goal):** Establecer los cimientos del sistema: entorno de desarrollo con túneles (Ngrok), esquema de seguridad (RBAC), registro de temporadas, categorías y grupos.
- **Enfoque de Trabajo:** En esta primera semana, el equipo no programa procesos de negocio, sino que levanta la infraestructura. Se asegura que la base de datos MariaDB conecte con Laravel, y se blinda el sistema para que todas las acciones dejen un rastro de auditoría automático desde el día uno.
- **Historias de Usuario:**
    - *Como Administrador*, quiero iniciar sesión de forma segura y tener un control de roles (RBAC) para restringir el acceso a los módulos.
    - *Como Administrador*, quiero configurar una "Temporada" y adjuntar su "Convocatoria" para definir las reglas del torneo.
    - *Como Administrador*, quiero crear Categorías (ej. Primera Fuerza, Veteranos) y dividirlas en Grupos.
- **Tareas Técnicas y Asignación:**
    - **[Dev 2 - Backend]:** Configurar base del proyecto en Laravel, conexión a MariaDB e implementar autenticación y middleware de Roles/Permisos (Spatie/laravel-permission). Crear migraciones para temporadas, categorias, grupos.
    - **[Dev 1 - Frontend]:** Diseñar el *layout* maestro (**Blade**) del panel administrativo (Sidebar, Topbar) y las vistas CRUD para Temporadas y Categorías.
    - **[Dev 4 - QA/Deploy]:** Configurar **Ngrok/Cloudflare** Tunnels para exponer el localhost. Implementar **Laravel Storage** privado para subir los PDFs de las Convocatorias.
    - **[Dev 3 - Lógica]:** Desarrollar el ***AuditService*** para registrar la "Auditoría inmutable de cambios críticos" (quién, cuándo, IP, valor anterior, valor nuevo) sin saturar los controladores.
- **Criterios de Aceptación (DoD):** Login funcional. Las rutas protegidas rechazan accesos no autorizados (403). La tabla auditoria_logs registra automaticamente la creación de una temporada.

Sprint 2: Afiliación de Clubes, Equipos y Tarjetón Digital (RF-03)

- **Objetivo (Sprint Goal):** Gestionar el padrón de clubes, asociar equipos a los grupos y generar el documento oficial de afiliación (Tarjetón Digital).
- **Enfoque de Trabajo:** El foco está en el manejo de entidades relacionales complejas y la generación de documentos. El Frontend trabajará en formularios dinámicos, mientras la lógica central abordará la regla del "reseteo histórico" cuando se venda una franquicia.
- **Historias de Usuario:**
    - *Como Presidente de Club*, quiero registrar mi club, sus colores y directiva para afiliarme a la temporada.
    - *Como Delegado*, quiero inscribir equipos en las categorías habilitadas.
    - *Como Secretario*, quiero generar el "Tarjetón Digital" en PDF con las firmas de los directivos.
- **Tareas Técnicas y Asignación:**
    - **[Dev 2 - Backend]:** Crear migraciones, Controladores y Form Requests (para validación - CRUD) para clubes, equipos, directivos_club.
    - **[Dev 1 - Frontend]:** Construir formularios de alta de clubes y equipos con selectores dinámicos (JS/Ajax) que filtren grupos por temporada.
    - **[Dev 4 - QA/Deploy]:** Integrar librería (barryvdh/laravel-dompdf o Browsershot) para renderizar el "Tarjetón Digital" y configurar su descarga protegida.
    - **[Dev 3 - Lógica]:** Programar la lógica del "Reseteo Histórico": si se sube una carta de venta de franquicia, archivar el club anterior e instanciar uno nuevo desde cero.
- **Criterios de Aceptación (DoD):** El Tarjetón Digital se genera en PDF con diseño formal. La venta de franquicia desvincula correctamente el historial deportivo previo.

Sprint 3: Registro de Jugadores, Documentos y Regla de Salto (RF-04)

- **Objetivo (Sprint Goal):** Completar el *roster* mediante la creación de la "Persona", carga de sus documentos verificatorios y validación estricta de elegibilidad deportiva.
- **Enfoque de Trabajo:** Este es un sprint de alta criticidad en seguridad de la información. Se trabajará en el almacenamiento cifrado de INEs y CURPs (LFPDPPP). Además, Dev 3 programa la primera restricción matemática fuerte: que un jugador no pueda saltar dos categorías.
- **Historias de Usuario:**
    - *Como Delegado*, quiero registrar jugadores subiendo su INE o CURP (según categoría).
    - *Como Comisión Infantil*, quiero validar la edad de los jugadores contra su CURP.
    - *Como Sistema*, quiero bloquear que un jugador se inscriba con un salto mayor a 1 categoría superior inmediata dentro de su club.
- **Tareas Técnicas y Asignación:**
    - **[Dev 2 - Backend]:** Migraciones para personas, jugadores_equipos y documentos_persona. Controladores para gestionar el padrón.
    - **[Dev 1 - Frontend]:** Interfaz de "Expediente del Jugador" mostrando el listado de documentos subidos y el estado mediante insignias visuales (Pendiente, Activo, Rechazado).
    - **[Dev 4 - QA/Deploy]:** Escribir *PHPUnit Feature Tests* para probar la regla de salto. Lógica de subida segura de fotos, INEs y CURPs al storage/app/private. Proteger endpoints de visualización de imágenes.
    - **[Dev 3 - Lógica]:** Desarrollar el servicio **ReglaSaltoCategoria**: validar que matematicamente **(Jerarquia_Destino - Jerarquia_Origen) <= 1** y que el club_id coincida.
- **Criterios de Aceptación (DoD):** Subir documentos confidenciales sin exponerlos en carpetas públicas (public/). El motor debe rechazar un salto de 2 categorías devolviendo un error 422 Unprocessable Entity con el mensaje normativo

Sprint 4: Canchas, Logística y Programación Manual (RF-05, RF-06)

- **Objetivo (Sprint Goal):** Establecer los recursos físicos (canchas) y habilitar la programación manual de una jornada deportiva.
- **Enfoque de Trabajo:** Se preparan los insumos para el motor de *Fixture*. El equipo se enfoca en resolver conflictos de calendario básicos (evitar solapamiento de horarios en la misma cancha).
- **Historias de Usuario:**
    - *Como Programador*, quiero gestionar el padrón de las 40+ canchas (U.D.C. y particulares), sus disponibilidades y bloqueos.
    - *Como Programador*, quiero crear una Jornada y asignar manualmente partidos especificando local, visitante, cancha y hora.
- **Tareas Técnicas y Asignación:**
    - **[Dev 2 - Backend]:** Migraciones para canchas, bloqueos_cancha, jornadas, partidos. Endpoints y Policies para el CRUD.
    - **[Dev 1 - Frontend]:** Panel tipo calendario o grilla (Kanban) para listar partidos programados. Pantallas para registrar bloqueos de canchas.
    - **[Dev 4 - QA/Deploy]:** Implementar subida y validación del formato PDF de "Préstamo de cancha".
    - **[Dev 3 - Lógica]:** Validación de estados del partido (Borrador -> Programado -> Confirmado) y prevención básica etricta de doble programación (que una cancha no se ocupe a la misma hora).
- **Criterios de Aceptación (DoD):** El sistema arroja un error visual claro si se intenta programar un partido en una cancha bloqueada por lluvia o con préstamo vencido.

Sprint 5: Motor de Fixture Automático (RF-07)

- **Objetivo (Sprint Goal):** Automatizar el trabajo pesado de calendarización resolviendo restricciones matemáticas de enfrentamientos y horarios.
- **Enfoque de Trabajo:** Sprint centrado fuertemente en Algoritmia. Dev 3 y Dev 2 se aíslan para crear un Constraint Engine que evalúe todas las variables, mientras Dev 4 somete el algoritmo a pruebas de estrés para asegurar que no crea "bucles infinitos".
- **Historias de Usuario:**
    - *Como Programador*, quiero que el sistema genere automáticamente los partidos cruzando equipos y asignando canchas.
    - *Como Sistema*, quiero respetar restricciones duras (sin solapamiento de horarios) y blandas (preferencias de cancha del club).
- **Tareas Técnicas y Asignación:**
    - **[Dev 2 - Backend]:** rear endpoints tipo API para invocar el motor de Fixture. Gestión de transacciones de base de datos para no guardar calendarios a medias.
    - **[Dev 1 - Frontend]:** Interfaz de evaluación que muestre el "Borrador" generado por el motor, destacando conflictos en rojo para que el Programador haga ajustes manuales
    - **[Dev 4 - QA/Deploy]:** QA profundo de los cruces generados; asegurar que todos los equipos juegan en formato Round-Robin sin repetir rival.
    - **[Dev 3 - Lógica]:** Programar el **FixtureConstraintEngine**.
        - 1) Generar cruces (Algoritmo de Berger),
        - 2) Asignar *slots* libres de las canchas,
        - 3) Evaluar restricciones de disponibilidad y preferencias
- **Criterios de Aceptación (DoD):** El algoritmo asigna un rol de juegos completo. Si hay más partidos que canchas disponibles, el sistema reporta el déficit sin colapsar.

Sprint 6: Cédula Arbitral Digital y Registro de Resultados (RF-08)

- **Objetivo (Sprint Goal):** Reemplazar el papel en la cancha mediante una app web responsiva (*Mobile-First*) para el cuerpo arbitral y procesar el resultado.
- **Enfoque de Trabajo:** Cambio de enfoque a la Experiencia de Usuario (UX) en dispositivos móviles. La vista de captura de cédula debe ser infalible (botones grandes, validación rápida) porque se usará al aire libre bajo el sol.
- **Historias de Usuario:**
    - *Como Árbitro*, quiero acceder desde mi celular, ver mi partido asignado y cotejar las identidades (fotos) de los jugadores.
    - *Como Árbitro*, quiero registrar goles, tarjetas, incidencias y recabar las firmas digitales de los capitanes al finalizar.
- **Tareas Técnicas y Asignación:**
    - **[Dev 2 - Backend]:** Migraciones para cedulas_arbitrales y eventos_partido. Lógica estricta de transición de estado del partido (En Juego -> Finalizado).
    - **[Dev 1 - Frontend]:** Diseñar vista estricta para móviles en Blade/Tailwind: botones grandes para sumar goles, *checkboxes* para alinear, integración de *canvas* JS para las firmas de capitanes.
    - **[Dev 4 - QA/Deploy]:** Asegurar y certificar el funcionamiento del Pad de Firmas en pantallas táctiles (iOS/Android).
    - **[Dev 3 - Lógica]:** Bloquear el envío de la cédula si faltan firmas o si se intenta alinear a un jugador que está "Suspendido" o "Inelegible" o si el marcador general no coincide con la suma atómica de eventos (goles).
- **Criterios de Aceptación (DoD):** El árbitro registra el resultado (ej. 2-1) y la cédula convierte y guarda la firma del Canvas en Base64/PNG, guardándola en el Storage. El partido pasa a estado "Finalizado".

Sprint 7: Tablas, Estadísticas, Disciplina y 5 Defaults (RF-09, RF-10)

- **Objetivo (Sprint Goal):** Procesar las secuelas del partido: recalcular estadísticas automáticamente, auditar incomparecencias y dictaminar castigos.
- **Enfoque de Trabajo:** Los eventos de la jornada provocan "Efectos Secundarios". El equipo implementa *Triggers* o *Jobs* en Laravel para que, al guardar una cédula, en segundo plano se calculen los puntos y se evalúen los castigos sin intervención humana.
- **Historias de Usuario:**
    - *Como Sistema*, quiero calcular automáticamente la Tabla de Posiciones y la Tabla de Goleo tras confirmar un resultado.
    - *Como Tribunal de Penas*, quiero ver las incidencias de las cédulas y proponer sanciones basadas en el Catálogo de Infracciones.
    - *Como Sistema*, quiero dar de baja automáticamente a un equipo al sumar 5 defaults.
- **Tareas Técnicas y Asignación:**
    - **[Dev 2 - Backend]:** Migraciones para tablas_posiciones, expedientes_disciplinarios, sanciones y recursos_inconformidad.
    - **[Dev 1 - Frontend]:** Vistas interactivas para las tablas de clasificación y el panel del Tribunal de Penas para cerrar resoluciones y generar el Boletín de Castigados.
    - **[Dev 4 - QA/Deploy]:** Escribir pruebas unitarias (QA exhaustivo) sobre el recálculo matemático de puntos, Goles a Favor (GF), Goles en Contra (GC) y Diferencia (DG).
    - **[Dev 3 - Lógica]:**
        - 1) Crear RecalcularTablaService evaluando empates.
        - 2) Implementar BajaCincoDefaultsService usando Laravel Observers para vigilar el contador y expulsar al equipo en el 5to default.
- **Criterios de Aceptación (DoD):** El equipo con 5 incomparecencias queda inhabilitado automáticamente. La tabla se actualiza en tiempo real sin requerir intervención humana.

Sprint 8: Tesorería, Fases Finales y Asambleas (RF-11, RF-13, RF-14)

- **Objetivo (Sprint Goal):** Concluir las operaciones contables, administrativas y gestionar el cierre de la temporada deportiva (Liguilla).
- **Enfoque de Trabajo:** Consolidación de procesos administrativos y cortes deportivos. El equipo desarrolla la lógica de "transición de fase" (de Fase Regular a Eliminatorias) generando visualmente el árbol del torneo.
- **Historias de Usuario:**
    - *Como Tesorero*, quiero registrar cobros de inscripciones, arbitrajes ($125) y mantener el control de adeudos.
    - *Como Secretario*, quiero registrar las asistencias y acuerdos de asamblea de delegados.
    - *Como Comité Ejecutivo*, quiero generar las llaves de Liguilla, Campeón de Campeones y procesar ascensos/descensos.
- **Tareas Técnicas y Asignación:**
    - **[Dev 2 - Backend]:** Migraciones y Controladores de cargos_financieros, pagos, asambleas y llaves_liguilla.
    - **[Dev 1 - Frontend]:** Vistas de estado de cuenta por club y panel interactivo de llaves eliminatorias (Brackets CSS/JS).
    - **[Dev 4 - QA/Deploy]:** Validar lógica de pagos parciales/totales y cambio de estado del adeudo.
    - **[Dev 3 - Lógica]:** Lógica de corte de Fase Regular: algoritmo para extraer los "N" primeros lugares de la tabla general y sembrarlos en la estructura de Liguilla (1 vs 8, 2 vs 7, etc.).
- **Criterios de Aceptación (DoD):** Un equipo clasificado a liguilla se inserta correctamente en su Bracket. El tesorero emite recibos de pago y los adeudos desaparecen de las alertas.

Sprint 9: Portal Público, Reportes PDF, Notificaciones y Cierre (RF-12, RF-15, RF-16)

- **Objetivo (Sprint Goal):** Dar cara al usuario final (afición), automatizar avisos y desplegar el sistema a producción.
- **Enfoque de Trabajo:** Pulido, optimización y salida a la nube. El equipo cierra los endpoints para el público, configura los servicios de correo SMTP y migra la infraestructura local hacia la solución de *hosting* final para la demostración.
- **Historias de Usuario:**
    - *Como Aficionado*, quiero entrar al portal web público (sin login) y ver roles de juegos, tablas, goleadores y castigados.
    - *Como Secretario*, quiero exportar la programación y el boletín a PDF para enviarlos a la prensa.
    - *Como Sistema*, quiero notificar por correo a los delegados si cambia el horario de su partido.
- **Tareas Técnicas y Asignación:**
    - **[Dev 2 - Backend]:** Consolidar API interna (Eager Loading para rendimiento) para alimentar el frontend público de solo lectura (sin endpoints de edición).
    - **[Dev 1 - Frontend]:** Maquetar la Landing Page del Portal Público (Móvil y Desktop) cuidando diseño, UX y rendimiento (Core Web Vitals).
    - ~~**[Dev 4 - QA/Deploy]:** Configurar servicio SMTP en Laravel (Mailables / Jobs). Optimizar el VPS, migrar BD MariaDB/PostgreSQL a producción, remover túneles de desarrollo y asociar el dominio web oficial.~~
    - **[Dev 3 - Lógica]:** Asegurar que las vistas PDF crucen correctamente toda la data relacional y aplicar el filtro de privacidad de la LFPDPPP en el portal público (ocultando datos sensibles).
- **Criterios de Aceptación (DoD):** Portal público online y responsivo (HTTPS). Sistema desplegado, correos asíncronos enviados exitosamente. Demostración Final lista para el cliente académico.

Parte 3: Base de Datos Completa (Esquema Relacional)

Con base en la arquitectura definida[cite: 11, 100], esta es la estructura relacional normalizada y ampliada para soportar PostgreSQL/MariaDB y satisfacer todas las Áreas Funcionales del ERS:

1. Core, Gobernanza y Portal Público

- **ligas**: id (PK), nombre, logo_url, estado.
- **usuarios: id (PK), email, password_hash, rol (ENUM), club_id (FK_null).**
- **auditoria_logs**: id (PK), usuario_id (FK), accion, tabla_afectada, registro_id, valores_anteriores (JSONB), valores_nuevos (JSONB), ip, created_at.
- **asambleas**: id (PK), fecha, acta_pdf_url, acuerdos (TEXT).
- **comunicados_noticias**: id (PK), liga_id (FK), titulo, contenido (TEXT), estado (Borrador/Publicado), publicado_en.
- **registro_notificaciones**: id (PK), destinatario_email, asunto, tipo_aviso, estado (Enviado/Fallido), fecha_envio. *(Cumple el RF-15.4)*.

2. Estructura Deportiva

- **temporadas**: id (PK), liga_id (FK), nombre, fecha_inicio, fecha_fin, estado.
- **convocatorias**: id (PK), temporada_id (FK), archivo_pdf, reglas_versionadas (JSONB).
- **categorias**: id (PK), nombre, edad_min, edad_max, jerarquia_salto (INT).
- **grupos**: id (PK), categoria_id (FK), temporada_id (FK), nombre.
- **fases_torneo**: id (PK), temporada_id (FK), tipo (Regular, Liguilla, Consolacion, CampeonDeCampeones).

3. Entidades Participantes y Documentos

- **clubes**: id (PK), nombre, logo, colores, fecha_afiliacion, estado.
- **directivos_club**: id (PK), club_id (FK), nombre, cargo, firma_digital_url.
- **historial_ventas_club**: id (PK), club_id (FK), propietario_anterior, propietario_nuevo, carta_venta_pdf, fecha_venta.
- **equipos**: id (PK), club_id (FK), grupo_id (FK), nombre, partidos_default_acumulados (INT, default 0).
- **personas**: id (PK), nombre, apellidos, fecha_nacimiento, curp.
- **documentos_persona**: id (PK), persona_id (FK), tipo (INE/CURP/Foto), ruta_storage, validado_en.
- **jugadores_equipos (Roster)**: id (PK), persona_id (FK), equipo_id (FK), dorsal, estado_elegibilidad (Activo, Inelegible, Suspendido).
- **arbitros**: id (PK), persona_id (UNIQUE FK), colegio_pertenencia, tarifa_por_partido, estado. *(Administración del padrón arbitral)*.

4. Logística y Programación

- **canchas**: id (PK), nombre, direccion_ubicacion, capacidad_partidos_consecutivos, estado.
- **bloqueos_cancha**: id (PK), cancha_id (FK), fecha_inicio, fecha_fin, motivo (Lluvia/Mantenimiento).
- **prestamos_cancha**: id (PK), cancha_id (FK), club_solicitante_id (FK), formato_prestamo_pdf, vigencia.
- **preferencias_equipo**: id (PK), equipo_id (FK), cancha_id_preferida (FK), dia_preferido, hora_preferida.
- **jornadas**: id (PK), grupo_id (FK), numero_jornada, fecha_inicio, fecha_fin.
- **partidos**: id (PK), jornada_id (FK), equipo_local_id (FK), equipo_visitante_id (FK), cancha_id (FK_null), arbitro_id (FK_null), fecha_partido, hora_partido, estado (ENUM: borrador, programado, confirmado, en juego, finalizado, suspendido).

5. Operación, Arbitraje y Disciplina

- **cedulas_arbitrales**: id (PK), partido_id (UNIQUE FK), goles_local, goles_visitante, informe_arbitral (TEXT), firma_capitan_local, firma_capitan_visitante.
- **eventos_partido**: id (PK), cedula_id (FK), jugador_id (FK_null), tipo (Gol, Amarilla, Roja), minuto.
- **catalogo_articulos**: id (PK), codigo, descripcion (TEXT), castigo_base_partidos (INT).
- **expedientes_disciplinarios**: id (PK), cedula_id (FK), partido_id (FK), articulo_id (FK), estado (Abierto/Resuelto).
- **sanciones**: id (PK), expediente_id (FK), persona_id (FK), descripcion_sancion (TEXT), partidos_suspendidos, partidos_cumplidos, estado.
- **recursos_inconformidad**: id (PK), sancion_id (FK), club_apelante_id (FK), evidencia_pdf, resolucion (TEXT), estado.

6. Competencia (Fases Finales) y Finanzas

- **tablas_posiciones**: id (PK), grupo_id (FK), equipo_id (FK), pj, pg, pe, pp, gf, gc, dg, puntos.
- **llaves_liguilla**: id (PK), fase_id (FK), equipo_local_id (FK), equipo_visitante_id (FK), marcador_global_local, marcador_global_visitante, avanza_equipo_id (FK_null).
- **cargos_financieros**: id (PK), club_id (FK), equipo_id (FK_null), persona_id (FK_null), concepto (Inscripción/Arbitraje/Credenciales/Multa), monto, vencimiento, estado (Pendiente/Pagado/Vencido).
- **pagos**: id (PK), cargo_id (FK), monto, metodo_pago, referencia, recibo_url, validado_por_tesorero_id (FK).

Información adicional...

- **NGROK** → Despliegue del localhost de manera gratuita
- **Github** → Versionamiento y trabajo en equipo del proyecto
    - Uso de ramas
- **Jira** → Organización del Proyecto (Scrum; backlog y sprints)
- **MARIADB** → Gestor de Base de Datos
- **LARAVEL** → Framework de Desarrollo
    - Uso de Modelo-Vista-Controlador
    - https://www.freecodecamp.org/espanol/news/el-modelo-de-arquitectura-view-controller-pattern/

